

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch09_feature_engineering

AYuSh

Contents

Chapter 9: Feature Engineering & Data Preprocessing	3
---	---

Chapter 9: Feature Engineering & Data Preprocessing

Models get the glory; features decide the outcome. The same gradient-boosted trees that win Kaggle competitions lose them when fed raw columns, and most production wins attributed to "a better model" are actually a better representation of the same data. Interviewers know this, which is why feature engineering questions are less about memorizing transformations and more about *judgment under leakage risk*: nearly every preprocessing step — imputation, encoding, scaling, selection, resampling — involves fitting something to data, and fitting it to the wrong data (test rows, future rows, the labels themselves) manufactures offline scores that evaporate in production. Chapter 4 introduced leakage as a concept; this chapter is where it becomes muscle memory, because feature engineering is where leakage actually happens.

The chapter walks the outline's territory in order: missing values and the mechanism taxonomy (MCAR/MAR/MNAR) that decides which imputation is even valid; categorical encoding from one-hot through target encoding to learned embeddings, with the naive-target-encoding leak staged and measured (+8 points of pure fiction in Listing 2); scaling and why the choice of scaler is a statement about your outliers; feature selection's three families (filter, wrapper, embedded) judged against planted ground truth; class imbalance, where the honest experiment shows resampling is mostly *not* about ranking metrics — and SMOTE-before-split fabricates a 0.999 PR-AUC; outlier detection and treatment, including why univariate rules miss exactly the outliers that do the most damage; the classical nonlinearity-buying transforms (log, binning, interactions), each shown rescuing a linear model from a specific failure; and the high-cardinality problem, where one-hot, hashing, frequency, and smoothed target encoding fight it out on a 900-category feature. Throughout, one rule organizes everything: **fit every transform on training data only, inside the cross-validation loop, ideally inside a Pipeline** — and every listing that violates it does so deliberately, with the damage measured.

Missing values and imputation

Before choosing an imputation strategy, ask *why* the values are missing — the answer determines what is statistically legitimate. The taxonomy (Rubin's) has three levels.

MCAR (missing completely at random): missingness is independent of everything — a sensor dropped packets. Any reasonable imputation is unbiased; you lose only efficiency.

MAR (missing at random): missingness depends on *observed* columns — older customers skip the income field, and age is recorded. Model-based imputation that conditions on the observed columns (iterative, KNN) can recover the structure. **MNAR**

(missing not at random): missingness depends on the *missing value itself* or on

unobserved facts — high earners decline to state income, defaulters hide debts. No imputation computed from observed data can be unbiased, because the observed distribution of that column is systematically censored. What *does* work under MNAR is refusing to pretend: **the missingness itself is a feature**. Listing 1 stages this: when holes are punched in a label-dependent way, mean/median/KNN/iterative imputation all cluster around 0.82–0.83 accuracy, while simply appending a binary missing-indicator column jumps to 0.862 — the indicator *is* the signal — and a model that handles NaN natively hits 0.923.

The strategies, in escalating sophistication. **Deletion** (drop rows or columns): defensible only under MCAR with few holes; under MAR/MNAR it biases the sample toward the kind of rows that answer questions. **Mean/median imputation**: fills with a constant; median resists skew and outliers. Cheap, but it shrinks the column's variance, distorts its correlations with everything else (imputed points sit on a horizontal line in any scatter plot), and under MNAR fills censored holes with the average of the *uncensored* — exactly wrong. **KNN imputation**: fill from the k nearest rows in the observed columns — respects local structure, costs $O(n^2)$ distances, inherits every scaling caveat of Chapter 6. **Iterative (model-based) imputation** (MICE-family): round-robin regress each incomplete column on all others, iterate to convergence — the strongest general-purpose option under MAR, and Listing 1's MCAR winner among imputers (0.809 vs mean's 0.790). **Missing-indicator augmentation**: add the mask as a column *alongside* any fill — nearly free, and the only honest play under suspected MNAR. **Native handling**: modern gradient boosting (XGBoost, LightGBM, HistGradientBoosting) treats NaN as a value, learning per-split which side missing rows go — Listing 1's overall winner in both regimes (0.877 MCAR, 0.923 MNAR), which is why "impute nothing, use HGB" is a legitimate answer, not an evasion.

Two leakage traps close the section. First, **fit imputers on training data only**: a mean computed over all rows includes test rows — mild in practice, but a pipeline violation that generalizes badly to deployment, where the "test set" hasn't arrived yet. Second, **impute after splitting, inside the CV loop** — an imputer fit on the full training set before cross-validation leaks each validation fold into the statistics used to fill it. Both are automatic if imputation lives in a `sklearn Pipeline`; both are hand-rolled bugs otherwise. Interview closer: never silently impute the target — rows with missing y are unlabeled data, not imputation candidates.

Encoding categorical features

Models eat numbers; categories must become numbers without acquiring false arithmetic. The choice of encoding is a choice about *what structure you assert*, and asserting wrong structure is the classic junior mistake.

Label / integer encoding maps categories to arbitrary integers (red=0, green=1, blue=2). For tree models this is merely awkward (a tree can carve $\{0,2\}$ vs $\{1\}$ in two

splits); for linear models and anything distance-based it is wrong — it asserts blue = 2×green, an order and a magnitude that don't exist. Reserve integer codes for **ordinal encoding**, where the order is real (S < M < L < XL; ratings; education levels) and you assign integers *in that order* — asserting equal spacing, which is itself an assumption to flag out loud.

One-hot encoding creates one binary column per category — the default for nominal categories with modest cardinality. Costs and caveats: dimensionality grows linearly with cardinality (the high-cardinality section below is the escape plan); the columns are perfectly multicollinear with the intercept (the "dummy variable trap" — drop one column for unregularized linear models; keep all for regularized or tree models, where it's harmless and drop-one actually distorts regularization symmetry); trees fragment — a 900-category one-hot gives each split a single yes/no on a single rare category, so depth is spent enumerating instead of generalizing; and unseen test categories need an explicit policy (`handle_unknown="ignore"` gives the all-zeros row — decide, don't discover).

Target encoding (mean encoding) replaces each category with the mean of the target over that category's rows — one dense column carrying exactly the signal a model wants. It is simultaneously the most powerful classical encoding and the most dangerous, because the encoding is *computed from labels*. Done naively — category means over all rows, including the row being encoded and the test set — each row's own label contaminates its feature. Listing 2 makes the fraud precise: a 2,000-category column of **pure noise** (three rows per category, zero true signal) plus one real feature. Honest model on the real feature: test 0.688. Add naively-encoded noise categories: test **0.780** — the model looks 8 points better *because test labels leaked into test features through the category means*. Out-of-fold encoding (each row encoded using means computed on other folds; test encoded with train-only means) returns 0.695 — the truth, give or take fold noise. The production forms add **smoothing**: shrink each category's mean toward the global mean in proportion to its count, $\tilde{y}_c = \frac{n_c \bar{y}_c + m \bar{y}}{n_c + m}$, so a 3-row category contributes mostly prior — this is an empirical-Bayes move, and it is what `category_encoders.TargetEncoder` and CatBoost's ordered target statistics implement (CatBoost encodes each row using only rows *before it* in a random permutation — leakage-proof by construction).

Embeddings are the deep-learning answer (Chapter 12 builds them): map each category to a learned dense vector, trained end-to-end with the model. They shine when cardinality is huge *and* categories have latent similarity structure worth learning (users, products, words), and they transfer across tasks. For tabular work with classical models, they're usually overkill — but "entity embeddings of categorical variables" is a name worth dropping, and the conceptual bridge (an embedding layer *is* a one-hot times a learned matrix) is a favorite interview connection. Cheap intermediate options that need no labels: **frequency encoding** (category → its count; surprisingly effective when popularity correlates with the target) and **hashing** (below, in high-cardinality).

Scaling and normalization

Scaling exists because many algorithms read *distance* or *gradient* meaning into raw feature magnitudes. Who cares: KNN, k-means, SVMs (kernel distances), PCA (variance directions), regularized linear models (a penalty $\lambda\|w\|^2$ taxes coefficients of small-scale features harder — Chapter 5), and neural networks (conditioning of the loss surface). Who doesn't: trees and tree ensembles — splits are order-based, and any monotone transform of a feature leaves its split structure unchanged. Listing 3 supplies the disaster number: one feature on a $500\times$ scale drops unscaled KNN to 0.579 (barely above chance) while any scaler restores ~ 0.88 — the big feature owned every distance, and five informative features were reduced to spectators.

The three standard scalers are three different claims about your data. **Standardization** (z-score): subtract mean, divide by standard deviation — the default; produces mean 0, variance 1; assumes no gross outliers, because both mean and especially σ are outlier-sensitive. **Min-max normalization**: map to $[0, 1]$ by the observed range — bounded output (nice for NNs and image-style inputs), but the *range* is the most outlier-fragile statistic there is: one absurd value compresses everyone else into a sliver. **Robust scaling**: subtract median, divide by IQR — immune to anything outside the middle 50%. Listing 3 quantifies the choice: with 2% gross outliers planted in the big feature, standard and min-max scaling leave the inliers' IQR at 0.031 and 0.0036 respectively — the outliers inflated the divisor, crushing the working scale of the real data $\sim 50\text{--}300\times$ below the other features' — while robust scaling holds $\text{IQR} = 1.0$ by construction. Accuracy differences stay modest here because KNN with 15 neighbors is forgiving, but the mechanism is the interview point: *a scaler fit on contaminated statistics silently re-weights features*.

Housekeeping that separates candidates: **fit the scaler on train, transform test with train's parameters** — scaling with test-set statistics is leakage (mild, but it's the same pipeline discipline as everything else); scale *inside* CV via Pipeline; sparse data wants `MaxAbsScaler` or no centering (subtracting a mean densifies sparsity); and **normalization** in the sklearn sense (`Normalizer`) is a different operation entirely — it rescales each *row* to unit norm (for cosine-similarity text vectors), not each column, and confusing the two is a small but telling error.

Feature selection

Feature selection buys three things: generalization (noise features are variance a model must overcome), economics (fewer features = cheaper inference, faster training, less to maintain in a feature store), and interpretability. The three families differ in what they consult. **Filter methods** score each feature against the target *without any model*: correlation, ANOVA F-statistic, mutual information (catches nonlinear dependence that correlation misses), chi-square for categorical pairs. Fast, scalable, embarrassingly parallel — and blind to interactions (a feature useless alone but decisive in combination

scores zero) and to redundancy (ten copies of one signal all score high; the filter happily keeps all ten). **Wrapper methods** search feature subsets by actually training and validating the model: forward selection, backward elimination, and **RFE** (recursive feature elimination: fit, drop the weakest by coefficient or importance, refit, repeat). Interaction-aware, model-specific — and expensive, $O(\text{model fits} \times \text{subsets explored})$, plus a real overfitting risk: enough subset search on one validation set fits the validation set. **Embedded methods** get selection as a fitting side effect: **L1/Lasso** drives coefficients exactly to zero (Chapter 5's geometry); tree-ensemble **feature importances** rank features by total impurity reduction or, better, **permutation importance** (shuffle a column, measure the score drop — model-agnostic, computed on validation data, immune to impurity importance's bias toward high-cardinality features).

Listing 4 judges all three against planted truth — 30 features: 6 informative, 6 redundant linear copies, 18 pure noise. Mutual-information filter, RFE, and random-forest importances each recover 8/8 real features in their top-8. The instructive miss is L1 at strong regularization: 6/10 real — it zeroed some *redundant* copies (correct behavior! Lasso picks one of a correlated group and discards the rest, Chapter 5's known instability) but let four noise features through. The lesson isn't "L1 is bad"; it's that each family has a characteristic failure signature — filters keep redundant copies, Lasso arbitrarily picks among them, importances split credit across them — and naming the signature is what the interviewer wants.

The section's leakage trap outranks the methods: **selecting features on the full dataset before cross-validation is leakage**, and it is the single most common way practitioners fabricate results on high-dimensional data. Select on all rows (test included), then CV: the selected features were chosen partly for their correlation with test labels, and with enough candidate features (genomics: 20,000 genes, 100 patients) you can reach near-perfect "cross-validated" accuracy on pure noise. The fix is mechanical: selection is a fitted transform, so it lives inside the Pipeline, refit per fold. Same rule, same reason, third appearance — it will not be the last.

Handling imbalanced data

With a 3% positive class, accuracy is a broken compass (97% by predicting "no" always — Chapter 10 dwells on metric choice; here we use PR-AUC). The menu: **class weights** (reweight the loss so each positive counts $1/\pi$ as much — no data change, one line: `class_weight="balanced"`); **random undersampling** (throw away majority rows until balanced — fast, loses information); **random oversampling** (duplicate minority rows — no information gained, overfitting risk as trees memorize repeated points); **SMOTE** (synthesize minority points by interpolating between minority nearest neighbors — the famous one); and **threshold moving** (train as-is, then place the decision threshold where the precision/recall trade-off belongs — often the rightest and least-used answer, because the default 0.5 threshold was never sacred).

Listing 5's honest experiment deflates the folklore. On 3.9% positives with a random forest, judged by PR-AUC — a *ranking* metric: baseline 0.768, class weights 0.748, random oversampling 0.749, SMOTE 0.708, undersampling 0.670. **Nothing beats doing nothing.** The mechanism: resampling and reweighting change the *threshold geometry* and probability calibration — they shift where scores concentrate — but a ranking metric only cares about the *order* of scores, and the baseline forest already orders fine; meanwhile undersampling discarded 92% of the majority class (real information, real cost) and SMOTE's linear interpolations manufacture points on segments between true positives, which in a 12-dimensional space with class overlap plants synthetic positives in majority territory. The honest summary for interviews: *imbalance handling mostly buys you a usable default threshold and calibrated-ish hard predictions; if you evaluate with ranking metrics and tune the threshold downstream, the baseline plus threshold moving is very hard to beat.* Class weights remain the cheapest defensible default when hard predictions are needed; focal loss (Chapter 14, born in object detection where imbalance is 1000:1) is the deep-learning-native variant — it down-weights already-easy examples so the loss concentrates on the hard minority.

Then the trap, which is worth more than the menu: **resample inside the CV loop, after splitting, training folds only** — never before. SMOTE before the split interpolates between minority points that will land in *different* sides of the split, so synthetic training points are literally blends of test rows; and the balanced "test set" no longer resembles the 3% world the model will face. Listing 5 measures the fantasy: PR-AUC **0.999** for SMOTE-before-split vs 0.708 done honestly. Any pipeline showing a suspiciously beautiful number on imbalanced data should be audited for exactly this bug first — it is epidemic in published applied-ML work. (`imblearn.pipeline.Pipeline` exists precisely so samplers can sit inside CV and apply only to training folds.)

Outlier detection and treatment

An outlier is a point that some model of "normal" says doesn't belong; the operational questions are *which model of normal* and *what to do about it*. Detection rules escalate in dimension-awareness. **Univariate rules:** z-score ($|z| > 3$ under approximate normality — but mean and σ are themselves outlier-inflated, so gross outliers mask each other; the MAD-based robust z-score fixes this) and **Tukey fences** (outside $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ — the boxplot whiskers; robust and distribution-lite). Both examine one column at a time. **Multivariate detectors** examine joint structure: Mahalanobis distance (elliptical normality), and Chapter 8's trio — Isolation Forest, LOF, one-class SVM — which need no distributional story at all.

Listing 6 stages why the distinction matters: 2% of rows corrupted at **high leverage** — extreme x paired with wrong y . Each corrupted value is unremarkable *marginally* (an x of 8 is just a big x ; a y near 0 is a typical y), so the univariate rules collapse: z-score on y flags 1/40 corrupted rows, IQR fences 2/40. Isolation Forest, seeing the *joint* (x, y) — where these points sit far from the data's line — catches 32/40. This is the general truth:

the outliers that damage models most are precisely the ones only visible jointly, because leverage is a relationship between columns, not a property of one.

The same listing measures treatment. The corruption drags OLS's slope from the true 2.0 to 0.897 — high-leverage points grab the regression line with force proportional to their distance from the x-center (Chapter 5's leverage). Options, with their measured recoveries: **winsorizing** x at the 1st/99th percentiles (clip, don't delete) manages only 0.941 — it pulls the extreme x's inward but *keeps the wrong y's*, now planted at moderate leverage: winsorizing treats a value, not a row, and a corrupted row stays corrupted. **Deleting flagged rows** recovers 1.668; **Huber loss** — Chapter 5's robust regression, quadratic near zero, linear in the tails, so gross residuals get bounded influence — recovers 1.759 with no detection step at all. Neither fully reaches 2.0 (the detector missed 8 corrupted rows; honesty in numbers). The judgment call interviewers probe: deletion is right when flagged points are *errors* (sensor glitches, joined-wrong rows) and dangerous when they're *real but rare* (a fraud model deleting its own fraud cases; the largest customer in a revenue model). When you can't know, prefer treatments that bound influence without destroying data: robust losses, robust scalers, rank/log transforms — and always ask where the outliers came from before choosing.

Binning, log transforms, and interaction features

These are the classical tools for buying nonlinearity without leaving the linear-model world — each rescues a specific, nameable failure, and Listing 7 measures all three. **Log transforms** fix multiplicative structure and right skew: when the truth is log-linear (elasticities, dose-response, anything where effects compound), a linear fit on the raw skewed feature manages R^2 0.603 while the same model on $\log(x)$ reaches 0.973. Rules: log needs positive inputs ($\log1p$ for counts with zeros; signed-log or Yeo-Johnson for real-valued); Box-Cox generalizes to a *fitted* power transform; and the workhorse justification — compressing a heavy right tail so the model isn't dominated by its largest values — applies to incomes, prices, view counts, and almost everything human-generated.

Binning (discretization) converts a numeric feature into interval indicators, letting a linear model fit a step function: on a U-shaped truth, raw-x linear regression achieves R^2 -0.001 — *literally nothing*, since the U's two arms cancel any single slope — while 10 equal-width one-hot bins reach 0.889. Costs: information loss inside bins, arbitrary edges (quantile bins beat equal-width on skewed data), and discontinuities at boundaries; splines are the smooth upgrade. Trees bin implicitly, which is one reason trees need none of this section. **Interaction features** multiply columns so an additive model can see joint effects: on an XOR-style truth (class = whether x_1 and x_2 agree in sign), logistic regression on $[x_1, x_2]$ scores 0.493 — a coin flip, because neither feature *alone* carries any signal, the textbook definition of a pure interaction — while adding the single engineered column $x_1 \cdot x_2$ scores 0.998. `PolynomialFeatures` automates degree-2 expansion at quadratic column cost; domain knowledge (price $\times$ promotion, dose $\times$ age) beats blind expansion; and this is exactly the gap kernels (Chapter 6) and deep networks (Chapter 12) close automatically — a connection worth saying aloud.

High-cardinality categoricals

Zip codes, user IDs, merchant IDs, URLs: thousands to millions of categories, long-tailed, with new values arriving daily. One-hot is arithmetic suicide at this scale (a million columns; each tree split interrogating one rare category), so the alternatives trade off density, leakage risk, and vocabulary maintenance. Listing 8's testbed — 900 categories, ~9 rows each, every category carrying a real effect — keeps the comparison honest.

One-hot (900 columns): 0.727 — workable at 900, dead at 900,000. **Feature hashing** (32 columns): hash each category string into a fixed number of buckets, accept collisions — 0.625, the price of ~28 categories sharing each bucket's identity. The trade is explicit: no vocabulary, no state, no unseen-category problem (new strings hash like old ones), memory fixed in advance — bought with collision noise that grows as buckets shrink. The tool of choice for streaming/online systems and million-way text features. **Frequency encoding** (1 column): 0.619 — here popularity carries no signal by construction, so it merely separates categories into rough count-buckets; on real data where popularity correlates with the target (popular products, active users) it routinely embarrasses fancier options at zero leakage risk. **Smoothed target encoding** (1 column): 0.734 — the winner, matching one-hot's information in 1/900th the width, *because* smoothing shrinks each ~9-row category's mean toward the prior instead of trusting it raw. The ranking to internalize: when categories genuinely predict the target, properly-regularized target encoding is the classical best-in-class; hashing when the vocabulary is unbounded or memory is fixed; frequency as the free lunch to always try; embeddings (Chapter 12) when there's enough data to learn structure; and the **rare-category bucket** (collapse everything below a count threshold into "other") composes with all of the above. Every one of these must handle the unseen-category case explicitly — global mean for target encoding, zero for frequency, its hash for hashing — because in production the categories you never saw are guaranteed to arrive.

Code implementations

Every listing was executed as shown; outputs are real. The set implements the chapter: the imputation bake-off across missingness mechanisms, the naive-target-encoding leak measured against out-of-fold honesty, the unscaled-KNN disaster and the outlier-vs-scaler study, the three selection families judged against planted truth, the imbalance menu deflated by a ranking metric plus the SMOTE-before-split fantasy, leverage outliers invisible to univariate rules, the three nonlinearity-buying transforms each rescuing a linear model, and the four-way high-cardinality shootout.

Listing 1 — Imputation strategies, and why the missingness mechanism decides

Under MCAR the imputers are within a point of each other and iterative wins the imputer race. Under MNAR — holes punched by the label — the missing-indicator column beats every clever fill, and native-NaN gradient boosting beats everything twice.

```

"""Listing 1: imputation strategies -- and why the missingness mechanism decides."""
import warnings
warnings.filterwarnings("ignore")
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.experimental import enable_iterative_imputer # noqa
from sklearn.impute import SimpleImputer, KNNImputer, IterativeImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
X, y = make_classification(n_samples=3000, n_features=8, n_informative=6,
random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.33, random_state=0)

def punch_holes(X, y, mnar):
    """MCAR: random 25% holes. MNAR: holes depend on the label -- missingness IS
    signal."""
    Xm = X.copy()
    if mnar:
        p = np.where(y == 1, 0.45, 0.05) # positives lose feature 0 often
        mask = rng.random(len(X)) < p
        Xm[mask, 0] = np.nan
    else:
        mask = rng.random(X.shape) < 0.25
        Xm[mask] = np.nan
    return Xm

for regime, mnar in [("MCAR (random holes)", False), ("MNAR (label-driven)", True)]:
    Xm_tr, Xm_te = punch_holes(X_tr, y_tr, mnar), punch_holes(X_te, y_te, mnar)
    strategies = {
        "mean": SimpleImputer(strategy="mean"),
        "median": SimpleImputer(strategy="median"),
        "KNN": KNNImputer(n_neighbors=5),
        "iterative": IterativeImputer(max_iter=10, random_state=0),
        "mean+indicator": SimpleImputer(strategy="mean", add_indicator=True),
    }
    print(f"\n{regime}")
    for name, imp in strategies.items():
        pipe = make_pipeline(imp, StandardScaler(), LogisticRegression(max_iter=2000))
        pipe.fit(Xm_tr, y_tr)
        print(f" {name:<15}: {pipe.score(Xm_te, y_te):.4f}")
    hgb = HistGradientBoostingClassifier(random_state=0).fit(Xm_tr, y_tr)
    print(f" {'native (HGB)':<15}: {hgb.score(Xm_te, y_te):.4f} (no imputation at
all)")

```

```

MCAR (random holes)
mean          : 0.7899
median        : 0.7980
KNN           : 0.8000
iterative     : 0.8091
mean+indicator : 0.7960
native (HGB)  : 0.8768 (no imputation at all)

MNAR (label-driven)
mean          : 0.8293
median        : 0.8273
KNN           : 0.8232
iterative     : 0.8232
mean+indicator : 0.8616
native (HGB)  : 0.9232 (no imputation at all)

```

Listing 2 — Naive target encoding: the leak, measured

The categorical column is 2,000 categories of pure noise. Naive all-rows encoding 'improves' test accuracy by 8 points over the honest baseline — fiction manufactured by baking test labels into test features. Out-of-fold encoding returns the truth.

```

"""Listing 2: target encoding computed before the split -- an offline mirage,
measured."""
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, KFold

rng = np.random.default_rng(1)
n = 6000
cat = rng.integers(0, 2000, n)           # ~3 rows per category, NO real signal
x = rng.normal(0, 1, n)                 # the only true signal
y = ((x + rng.normal(0, 1, n)) > 0).astype(int)
df = pd.DataFrame({"cat": cat, "x": x})
tr, te = train_test_split(np.arange(n), test_size=0.33, random_state=1)

def score(X):
    m = RandomForestClassifier(200, random_state=1, n_jobs=-1).fit(X[tr], y[tr])
    return m.score(X[tr], y[tr]), m.score(X[te], y[te])

baseline = score(df[["x"]].values)
print(f"x alone           : train {baseline[0]:.3f} test {baseline[1]:.3f}")

# NAIVE target encoding: category -> mean(y) over ALL its rows (incl. this row's label)
means_all = pd.Series(y).groupby(df["cat"]).mean()
X_naive = np.c_[df["cat"].map(means_all), df["x"]]
trn, ten = score(X_naive)
print(f"target enc (naive) : train {trn:.3f} test {ten:.3f} <- inflated: test
labels leaked")

# OUT-OF-FOLD encoding: each row sees only other folds' means; test sees train means
enc = np.zeros(n)
gm = y[tr].mean()
for f_tr, f_val in KFold(5, shuffle=True, random_state=1).split(tr):
    means = pd.Series(y[tr[f_tr]]).groupby(df["cat"].iloc[tr[f_tr]]).mean()
    enc[tr[f_val]] = df["cat"].iloc[tr[f_val]].map(means).fillna(gm)
means_tr = pd.Series(y[tr]).groupby(df["cat"].iloc[tr]).mean()
enc[te] = df["cat"].iloc[te].map(means_tr).fillna(gm)
tro, teo = score(np.c_[enc, df["x"]])
print(f"target enc (OOF)   : train {tro:.3f} test {teo:.3f} <- honest")
print("\nThe category is pure noise -- yet naive encoding 'beats' the honest 0.695,")
print("because encoding on ALL rows bakes test labels into test features. That +8pt")
print("edge is an offline mirage: in production, new rows have no labels to leak.")

```

```

x alone           : train 1.000 test 0.688
target enc (naive) : train 1.000 test 0.780 <- inflated: test labels leaked
target enc (OOF)   : train 1.000 test 0.695 <- honest

```

The category is pure noise -- yet naive encoding 'beats' the honest 0.695, because encoding on ALL rows bakes test labels into test features. That +8pt edge is an offline mirage: in production, new rows have no labels to leak.

Listing 3 — Scaling: the unscaled disaster, and what outliers do to each scaler

One feature on a 500x scale reduces unscaled KNN to near-chance. With 2% gross outliers planted, standard and min-max scaling crush the inliers' working range (IQR 0.031 and 0.0036) while robust scaling holds it at 1.0 by construction.

```
"""Listing 3: scaling -- the unscaled disaster, and what outliers do to each scaler."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

rng = np.random.default_rng(2)
X, y = make_classification(n_samples=2000, n_features=6, n_informative=4,
random_state=2)
X[:, 0] *= 500 # one feature on a 500x scale
X[rng.choice(2000, 40), 0] += rng.normal(2e5, 3e4, 40) # plus 2% gross outliers
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.33, random_state=2)

# Other features have IQR ~ 1.3 after their own scaling; compare feature 0's
print(f"{'scaler':<9} {'KNN test acc':>13} {'IQR of scaled f0':>17}")
for name, sc in [("none", None), ("standard", StandardScaler()),
                 ("min-max", MinMaxScaler()), ("robust", RobustScaler())]:
    steps = ([sc if sc else []] + [KNeighborsClassifier(15)])
    acc = make_pipeline(*steps).fit(X_tr, y_tr).score(X_te, y_te)
    f0 = sc.fit_transform(X_tr[:, 0]) if sc else X_tr[:, 0]
    iqr = np.subtract(*np.percentile(f0, [75, 25]))
    print(f"{'name':<9} {'acc':>13.4f} {'iqr':>17.4f}")
print("\nUnscaled: the 500x feature owns every distance -- accuracy collapses.")
print("Standard and min-max let the 2% outliers inflate the divisor, shrinking the")
print("inliers' working scale ~50x below the other features'; robust (median/IQR)")
print("restores feature 0 to parity by construction (IQR = 1).")
```

scaler	KNN test acc	IQR of scaled f0
none	0.5788	980.2859
standard	0.8833	0.0310
min-max	0.8879	0.0036
robust	0.8727	1.0000

```
Unscaled: the 500x feature owns every distance -- accuracy collapses.
Standard and min-max let the 2% outliers inflate the divisor, shrinking the
inliers' working scale ~50x below the other features'; robust (median/IQR)
restores feature 0 to parity by construction (IQR = 1).
```

Listing 4 — Filter, wrapper, embedded selection vs planted truth

Thirty features: 6 informative, 6 redundant copies, 18 noise. Mutual information, RFE, and forest importances each recover 8/8 real features; L1 at strong regularization shows its signature failure — it discards redundant copies (by design) but admits four noise features.

```
"""Listing 4: feature selection -- filter, wrapper, embedded, judged against known
truth."""
import numpy as np
```

```

from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import (SelectKBest, mutual_info_classif, RFE)
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# 30 features: the FIRST 6 are informative, 6 are redundant copies, 18 pure noise
X, y = make_classification(n_samples=2000, n_features=30, n_informative=6,
                          n_redundant=6, shuffle=False, random_state=3)
Xs = StandardScaler().fit_transform(X)
truth = set(range(12))          # informative + their redundant copies

def report(name, chosen):
    chosen = [int(c) for c in chosen]
    hits = len(set(chosen) & truth)
    print(f"{name:<22}: {sorted(chosen)}  ({hits}/{len(chosen)} real)")

# FILTER: score each feature alone (fast, model-free, blind to interactions)
mi = SelectKBest(mutual_info_classif, k=8).fit(Xs, y)
report("filter (mutual info)", np.where(mi.get_support())[0])

# WRAPPER: recursive feature elimination around a model (slow, interaction-aware)
rfe = RFE(LogisticRegression(max_iter=2000), n_features_to_select=8).fit(Xs, y)
report("wrapper (RFE)", np.where(rfe.support_)[0])

# EMBEDDED: selection as a side effect of fitting
l1 = LogisticRegression(penalty="l1", C=0.05, solver="liblinear").fit(Xs, y)
report("embedded (L1)", np.where(np.abs(l1.coef_[0]) > 1e-6)[0])
rf = RandomForestClassifier(300, random_state=3, n_jobs=-1).fit(Xs, y)
report("embedded (RF top-8)", np.argsort(rf.feature_importances_)[::-1][:8])

```

```

filter (mutual info) : [0, 2, 3, 4, 5, 7, 9, 11] (8/8 real)
wrapper (RFE)       : [0, 1, 2, 3, 4, 5, 9, 11] (8/8 real)
embedded (L1)       : [0, 1, 2, 3, 5, 11, 16, 17, 19, 23] (6/10 real)
embedded (RF top-8) : [0, 1, 3, 4, 5, 8, 9, 11] (8/8 real)

```

Listing 5 — Imbalance: weights vs resampling vs SMOTE, and the SMOTE-before-split trap

Judged by PR-AUC — a ranking metric — nothing beats the untouched baseline: resampling reshapes threshold geometry, not ranking. SMOTE applied before the split interpolates test rows into training data and reports a fantasy 0.999.

```

"""Listing 5: class imbalance -- weights vs resampling vs SMOTE, and the SMOTE-before-split trap."""
import warnings
warnings.filterwarnings("ignore")
import numpy as np
from imblearn.over_sampling import SMOTE, RandomOverSampler
from imblearn.under_sampling import RandomUnderSampler
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=12000, n_features=12, n_informative=6,
                          weights=[0.97], flip_y=0.02, random_state=4) # 3%
positives
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.33,
                                          stratify=y, random_state=4)

def pr_auc(m, Xt=None, yt=None):

```



```

Xt, yt = (X_te, y_te) if Xt is None else (Xt, yt)
return average_precision_score(yt, m.predict_proba(Xt)[: , 1])

rf = lambda: RandomForestClassifier(300, random_state=4, n_jobs=-1)
print(f"positives: {y_tr.mean():.1%} of train")
print(f"baseline (as-is) : PR-AUC {pr_auc(rf().fit(X_tr, y_tr)):.3f}")
w = RandomForestClassifier(300, class_weight='balanced', random_state=4, n_jobs=-1)
print(f"class_weight=balanced : PR-AUC {pr_auc(w.fit(X_tr, y_tr)):.3f}")
for name, sampler in [("random undersample", RandomUnderSampler(random_state=4)),
                      ("random oversample", RandomOverSampler(random_state=4)),
                      ("SMOTE", SMOTE(random_state=4))]:
    Xr, yr = sampler.fit_resample(X_tr, y_tr)
    print(f"{name:<22}: PR-AUC {pr_auc(rf().fit(Xr, yr)):.3f}")

# THE TRAP: SMOTE before the split -- synthetic points interpolate future test rows
Xs, ys = SMOTE(random_state=4).fit_resample(X, y)
Xs_tr, Xs_te, ys_tr, ys_te = train_test_split(Xs, ys, test_size=0.33, random_state=4)
m = rf().fit(Xs_tr, ys_tr)
print(f"\nSMOTE BEFORE split : PR-AUC {pr_auc(m, Xs_te, ys_te):.3f} <- fantasy
number")

```

```

positives: 3.9% of train
baseline (as-is) : PR-AUC 0.768
class_weight=balanced : PR-AUC 0.748
random undersample : PR-AUC 0.670
random oversample : PR-AUC 0.749
SMOTE : PR-AUC 0.708

SMOTE BEFORE split : PR-AUC 0.999 <- fantasy number

```

Listing 6 — Leverage outliers: invisible marginally, fatal jointly

Corrupted rows pair extreme x with wrong y, so each value is unremarkable alone: univariate rules flag 1-2 of 40; Isolation Forest, seeing the joint distribution, flags 32. The corruption drags the OLS slope from 2.0 to 0.897; winsorizing the value barely helps (the row stays corrupted), while deletion and Huber loss recover most of the truth.

```

"""Listing 6: outliers -- detection rules compared, then treatment options measured."""
import warnings
warnings.filterwarnings("ignore")
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.linear_model import LinearRegression, HuberRegressor

rng = np.random.default_rng(7)
n = 2000
x = rng.normal(0, 1, n)
y = 2.0 * x + rng.normal(0, 1, n) # true slope = 2
out = rng.choice(n, 40, replace=False) # 2% corrupted rows...
x[out] = rng.normal(8, 1, 40) # ...at HIGH LEVERAGE (extreme x)
y[out] = rng.normal(0, 3, 40) # ...with wrong y
x_te = rng.normal(0, 1, 1000) # clean test set
y_te = 2.0 * x_te + rng.normal(0, 1, 1000)

# --- detection: three rules, judged against the known corrupted rows ---
z = np.abs((y - y.mean()) / y.std()) # z-score on y alone
q1, q3 = np.percentile(y, [25, 75])
fence = (y < q1 - 1.5 * (q3 - q1)) | (y > q3 + 1.5 * (q3 - q1)) # Tukey, y alone
iso = IsolationForest(contamination=0.02, random_state=7).fit(np.c_[x, y])
flags = {
    "z-score>3 (y only)": np.where(z > 3)[0],
    "1.5*IQR (y only)": np.where(fence)[0],
}

```

```

    "IsolationForest": np.where(iso.predict(np.c_[x, y]) == -1)[0],
}
truth = set(out.tolist())
for name, idx in flags.items():
    tp = len(set(idx.tolist()) & truth)
    print(f"{name:<18}: flagged {len(idx):>3}, real {tp}/40")

# --- treatment: what each choice does to the fitted slope and clean-test RMSE ---
def fit(xs, ys, model=None):
    m = (model or LinearRegression()).fit(xs[:, None], ys)
    rmse = np.sqrt(np.mean((m.predict(x_te[:, None]) - y_te) ** 2))
    return m.coef_[0], rmse

lo, hi = np.percentile(x, [1, 99])
keep = iso.predict(np.c_[x, y]) == 1 # drop IsolationForest flags
for name, (slope, rmse) in {
    "OLS, as-is": fit(x, y),
    "winsorize x 1/99": fit(np.clip(x, lo, hi), y),
    "delete flagged": fit(x[keep], y[keep]),
    "Huber loss": fit(x, y, HuberRegressor()),
}.items():
    print(f"{name:<18}: slope {slope:5.3f} clean-test RMSE {rmse:5.3f}")

```

```

z-score>3 (y only): flagged 5, real 1/40
1.5*IQR (y only) : flagged 13, real 2/40
IsolationForest : flagged 40, real 32/40
OLS, as-is      : slope 0.897 clean-test RMSE 1.493
winsorize x 1/99 : slope 0.941 clean-test RMSE 1.462
delete flagged  : slope 1.668 clean-test RMSE 1.063
Huber loss      : slope 1.759 clean-test RMSE 1.039

```

Listing 7 — Log, bins, interactions: three rescues of a linear model

Each transform targets a nameable failure: log-linear truth (R^2 0.603 raw, 0.973 logged), U-shaped truth (R^2 -0.001 raw — the arms cancel — 0.889 binned), and pure-interaction truth (coin-flip additive, 0.998 with the single product column).

```

"""Listing 7: log transforms, binning, interactions -- buying nonlinearity for linear models."""
import warnings
warnings.filterwarnings("ignore")
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import KBinsDiscretizer

rng = np.random.default_rng(5)
n = 4000

# (a) log transform: multiplicative truth, skewed feature
x_skew = rng.lognormal(0, 1, n) # heavy right tail
y_mult = 3 * np.log(x_skew) + rng.normal(0, 0.5, n) # truth is log-linear
tr, te = train_test_split(np.arange(n), test_size=0.33, random_state=5)
def r2(feats, target):
    m = LinearRegression().fit(feats[tr], target[tr])
    return m.score(feats[te], target[te])
print(f"skewed feature, raw : R^2 {r2(x_skew[:, None], y_mult):.3f}")
print(f"skewed feature, log : R^2 {r2(np.log(x_skew[:, None]), y_mult):.3f}")

# (b) binning: U-shaped truth defeats a line; bins buy a step function
x_u = rng.uniform(-3, 3, n)
y_u = x_u**2 + rng.normal(0, 0.7, n)

```



```

print(f"\nU-shaped truth, raw      : R^2 {r2(x_u[:, None], y_u):.3f}")
bins = KBinsDiscretizer(n_bins=10, encode='onehot-dense').fit(x_u[tr, None])
print(f"U-shaped truth, binned: R^2 {r2(bins.transform(x_u[:, None]), y_u):.3f}")

# (c) interactions: XOR-style truth invisible to additive models
x1, x2 = rng.normal(size=n), rng.normal(size=n)
y_xor = ((x1 * x2) > 0).astype(int)           # sign agreement decides the class
Xa = np.c_[x1, x2]
Xi = np.c_[x1, x2, x1 * x2]                   # add the product feature
def acc(X):
    m = LogisticRegression().fit(X[tr], y_xor[tr])
    return m.score(X[te], y_xor[te])
print(f"\nXOR truth, additive      : acc {acc(Xa):.3f} (coin flip)")
print(f"XOR truth, + x1*x2         : acc {acc(Xi):.3f} (one engineered column)")

```

```

skewed feature, raw      : R^2 0.603
skewed feature, log      : R^2 0.973

U-shaped truth, raw      : R^2 -0.001
U-shaped truth, binned: R^2 0.889

XOR truth, additive      : acc 0.493 (coin flip)
XOR truth, + x1*x2       : acc 0.998 (one engineered column)

```

Listing 8 — 900 categories: one-hot vs hashing vs frequency vs smoothed target

Every category carries real signal, ~9 rows each. Smoothed target encoding wins in one column (0.734), matching 900-column one-hot (0.727); hashing pays a measured collision tax (0.625); frequency encoding fails here by construction — popularity was built to carry no signal.

```

"""Listing 8: high-cardinality categoricals -- one-hot vs hashing vs frequency vs
smoothed target."""
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
from category_encoders import TargetEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction import FeatureHasher
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(6)
n, k = 8000, 900                                # 900 categories, ~9 rows each
cat = rng.integers(0, k, n)
effect = rng.normal(0, 1, k)                      # each category has a real effect
x = rng.normal(0, 1, n)
y = ((effect[cat] + x + rng.normal(0, 1, n)) > 0).astype(int)
cat_str = np.array([f"c{c}" for c in cat])
tr, te = train_test_split(np.arange(n), test_size=0.33, random_state=6)

def score(Xtr, Xte, note=""):
    m = RandomForestClassifier(300, random_state=6, n_jobs=-1).fit(Xtr, y[tr])
    return m.score(Xte, y[te])

# one-hot: 900 columns, ~9 ones each -- wide, sparse, fragmenting
oh = pd.get_dummies(pd.Series(cat_str))
X_oh = np.c_[oh.values, x]
print(f"one-hot ({oh.shape[1]} cols)      : acc {score(X_oh[tr], X_oh[te]):.3f}")

```

```
# hashing trick: fixed 32 columns, no vocabulary, collisions accepted
H = FeatureHasher(n_features=32, input_type="string")
Xh = np.c_[H.transform([[c] for c in cat_str]).toarray(), x]
print(f"hashing (32 cols)      : acc {score(Xh[tr], Xh[te]):.3f}")

# frequency encoding: category -> its count (cheap, target-free)
freq = pd.Series(cat_str[tr]).value_counts()
Xf = np.c_[pd.Series(cat_str).map(freq).fillna(0), x]
print(f"frequency (1 col)     : acc {score(Xf[tr], Xf[te]):.3f}")

# smoothed target encoding (category_encoders: fits on TRAIN only, shrinks rare cats)
tenc = TargetEncoder(cols=[0], smoothing=5.0)
Xt_tr = np.c_[tenc.fit_transform(pd.DataFrame(cat_str[tr]), y[tr]).values, x[tr]]
Xt_te = np.c_[tenc.transform(pd.DataFrame(cat_str[te])).values, x[te]]
print(f"target enc (1 col)    : acc {score(Xt_tr, Xt_te):.3f}")
```

```
one-hot (900 cols)      : acc 0.727
hashing (32 cols)       : acc 0.625
frequency (1 col)       : acc 0.619
target enc (1 col)      : acc 0.734
```

Pitfalls, comparisons and practical tips

The leakage ledger. Every fitted preprocessing step must be fit on training folds only, inside the CV loop. The chapter measured three violations: naive target encoding (+8 points of fiction, Listing 2), SMOTE before the split (0.708 → 0.999, Listing 5), and — from the literature — selection-before-CV, which can certify pure noise as near-perfectly predictive. Milder cousins (imputer/scaler statistics computed over test rows) follow the identical rule. One mechanical habit prevents all of it: transforms live in a `Pipeline`, and the `Pipeline` is what enters `cross_val_score`. If a preprocessing step touches labels — target encoding, SMOTE, supervised selection — treat it as maximally dangerous by default.

Encoding chooser.

Situation	Encoding	Watch for
Nominal, low cardinality (< ~30)	One-hot	Dummy trap for unregularized linear; unseen categories
True order exists	Ordinal (in order)	Equal-spacing assumption
High cardinality, signal in category	Smoothed / OOF target encoding	Leakage; rare categories need shrinkage
Unbounded vocabulary, streaming	Hashing	Collision noise; pick bucket count by memory
Popularity plausibly predictive	Frequency	Free to try, target-free
Huge cardinality + deep model	Learned embeddings	Needs data; Chapter 12

Situation	Encoding	Watch for
Long tail of rare values	Rare-category bucket first	Threshold is a hyperparameter

Scaler chooser. Standard by default; robust when outliers are real and staying; min-max when the downstream wants bounded inputs and the data is clean; MaxAbs/no-centering for sparse. Trees: none needed. Row-wise `Normalizer` is a different animal (unit-norm rows for cosine similarity) — don't confuse it with column scaling.

Imputation quick rules. Diagnose the mechanism before choosing the tool: plot missingness against other columns and against the target (a missingness-vs-target dependence is the MNAR alarm). Under suspected MNAR, add the indicator column — it costs nothing and is the only unbiased move available. Gradient boosting with native NaN handling is a legitimate complete answer. Never impute the target.

Imbalance quick rules. First fix the metric (PR-AUC / recall-at-precision, Chapter 10), then ask whether you need better *ranking* (rarely improvable by resampling — Listing 5) or better *hard predictions at a threshold* (fix by threshold moving or class weights). SMOTE is a legitimate tool that mostly matters for extreme scarcity of minority examples with simple decision boundaries — and it must live inside the CV loop via `imblearn`'s Pipeline.

Outlier judgment. Univariate rules miss leverage (Listing 6); check jointly when columns should covary. Ask provenance before treatment: error → delete or fix upstream; real-but-rare → keep, bound its influence (Huber, robust scaler, log/rank transform); the outlier IS the phenomenon (fraud, failures) → it's your positive class, not your trash. Winsorizing treats a value, not a row — a corrupted row keeps lying through its other columns.

Trees vs linear: who needs what. A useful compression of the whole chapter: tree ensembles are indifferent to monotone transforms, scaling, and binning, handle NaN natively (modern ones), and split on raw categoricals (LightGBM/CatBoost) — their feature engineering need is interactions across many variables (which depth supplies) and signal-bearing encodings for huge cardinalities. Linear models need everything: scaling for the penalty, log/bins/splines for nonlinearity, explicit interaction columns, one-hot or target encodings, and imputation. "Which model are you feeding?" is the first question of any feature-engineering answer.

Interview questions and answers

Q1. Define MCAR, MAR, and MNAR, and explain why the mechanism decides which imputation is even valid.

MCAR (missing completely at random): missingness is independent of everything — a dropped sensor packet. Any reasonable fill is unbiased; you lose only efficiency.

MAR: missingness depends on *observed* columns (older customers skip income, age is recorded), so model-based imputation conditioning on those columns (iterative, KNN) can recover the structure.

MNAR: missingness depends on the *missing value itself* or unobserved facts (high earners decline to state income). No imputation from observed data is unbiased, because the observed distribution of that column is systematically censored. Rubin's taxonomy isn't trivia — it tells you whether a fill can be honest at all, and the diagnostic is plotting missingness against other columns and the target.

Q2. Under suspected MNAR, what is the only unbiased move, and why?

Stop pretending you can fill the hole and instead make the *missingness itself a feature*: append a binary indicator column alongside any fill. Under MNAR the fact that a value is missing carries signal (defaulters hide debts), so the mask often out-predicts the imputed number. Listing 1 measures it: with label-driven holes, every imputer clusters at 0.82–0.83 accuracy while `mean + indicator` jumps to 0.862 — the indicator *is* the signal. It costs one nearly-free column and is the only move that doesn't bake the censoring bias into the fill.

Q3. Compare mean/median, KNN, and iterative (MICE) imputation on cost and assumptions.

Mean/median: fill with one constant (median resists skew); cheap, but it shrinks the column's variance and flattens its correlations — imputed points sit on a line in any scatter — and under MNAR fills censored holes with the uncensored average, exactly wrong. *KNN*: fill from the k nearest rows in the observed columns; respects local structure, costs $O(n^2)$ distances, inherits every scaling caveat. *Iterative / MICE*: round-robin regress each incomplete column on all others to convergence — the strongest general-purpose option under MAR (Listing 1's MCAR winner among imputers, 0.809 vs mean's 0.790). Sophistication buys MAR-validity, not MNAR-validity — nothing computed from observed data escapes censoring.

Q4. Where in the pipeline must imputation live, and what breaks if it doesn't?

Inside a `Pipeline`, fit on training folds only, refit per CV fold. A mean computed over all rows includes test rows; an imputer fit on the full training set before cross-validation leaks each validation fold into the statistics used to fill it. Both are silent optimism that evaporates in deployment, where "test rows" haven't arrived. The rule is mechanical and automatic once the imputer is a pipeline step; it's a hand-rolled bug otherwise — the same fit-on-train-only discipline that governs scaling, selection, and resampling.

Q5. Should you ever impute the target variable?

No. Rows with missing `y` are unlabeled data, not imputation candidates — inventing their labels manufactures signal from nothing and corrupts every downstream metric. Drop them from supervised training (or exploit them via semi-supervised methods that treat them *as* unlabeled), but never fill them with a model's guess and then train on that guess as if it were ground truth.

Q6. When is "don't impute at all, use gradient boosting" a legitimate answer rather than an evasion?

Whenever your model is modern gradient boosting (XGBoost, LightGBM, HistGradientBoosting), which treats NaN as a value and learns per-split which side missing rows go. It handles MCAR and MNAR structure without any fill: Listing 1's overall winner in both regimes (0.877 MCAR, 0.923 MNAR), beating every imputer. It sidesteps the leakage traps entirely because there's no fitted fill statistic to leak. The honest caveat: it only works because the tree can route missingness — a linear model or KNN still needs a real imputation strategy.

Q7. Label vs ordinal vs one-hot encoding — what structure does each assert?

Label/integer (red=0, green=1, blue=2) asserts an order *and* a magnitude — blue = 2×green — that don't exist; fine for trees (they can carve {0,2} vs {1}), wrong for anything linear or distance-based. *Ordinal* assigns integers *in a real order* (S<M<L<XL, education levels), asserting equal spacing — itself an assumption to flag aloud. *One-hot* asserts no order at all, one binary column per category — the correct default for nominal categories, at the cost of dimensionality. The junior mistake is asserting order where there is none; the senior move is naming exactly what arithmetic each encoding smuggles in.

Q8. Explain the dummy-variable trap. When do you drop a one-hot column and when do you keep all of them?

A full one-hot set is perfectly collinear with the intercept (the columns sum to 1), so an unregularized linear model has a non-identifiable coefficient — *drop one column* (the reference level). But keep *all* columns for regularized linear models and trees: with an L2/L1 penalty, drop-one distorts the regularization symmetry (the dropped level gets penalty-free treatment via the intercept), and trees don't care about collinearity at all. So the rule is conditional: drop-one only for unregularized linear; keep-all otherwise.

Q9. What is target encoding, and precisely how does the naive version leak?

Target (mean) encoding replaces each category with the mean of the target over that category's rows — one dense column carrying exactly the signal a model wants, and the strongest classical encoding for high cardinality. Naive computation takes category means over *all* rows, including the row being encoded and the test set, so each row's own label contaminates its feature. Listing 2 stages it on 2,000 categories of *pure noise* (three rows each): honest test AUC 0.688, naive 0.780 — eight points of fiction, because test labels leaked into test features through the category means. In production, new rows have no labels to leak, so the edge is a pure offline mirage.

Q10. How do out-of-fold encoding and smoothing fix target-encoding leakage?

Out-of-fold: encode each training row using means computed on *other* folds, and encode test with train-only means — Listing 2 returns 0.695, the truth give or take fold noise. *Smoothing* shrinks each category's mean toward the global mean in proportion to its count, $y_c = (n_c * \text{mean}_c + m * \text{mean_global}) / (n_c + m)$, so a 3-row category contributes mostly prior — an empirical-Bayes move that stabilizes rare categories. CatBoost's ordered target statistics go further: encode each row using only rows *before it* in a random permutation, leakage-proof by construction. `category_encoders.TargetEncoder` implements the smoothed form and fits on train only.

Q11. When are frequency encoding and the hashing trick the right call, and what do they cost?

Frequency encoding (category → its count) is free, label-free, and surprisingly effective when popularity correlates with the target; it fails when it doesn't — Listing 8's frequencies were built to carry no signal and it lands last at 0.619. *Hashing* maps categories into a fixed number of buckets with no vocabulary — ideal for unbounded/streaming cardinality and tight memory — but pays a measured collision tax (Listing 8: 32 buckets → 0.625, below one-hot's 0.727) because colliding categories share a column and blur their signals. Pick hashing when the vocabulary is open-ended; pick frequency when popularity is plausibly predictive and you want a target-free single column.

Q12. What are entity embeddings, and when are they worth the trouble over target encoding?

An embedding maps each category to a learned dense vector, trained end-to-end with a neural model (Chapter 12). They win when cardinality is huge *and* categories share latent similarity structure worth learning — users, products, words — and they transfer across tasks. For tabular work with classical models they're usually overkill; smoothed/OOF target encoding gets most of the signal in one column with no deep model. The interview bridge worth naming: an embedding layer *is* a one-hot times a learned matrix, so it's the same lookup as one-hot encoding but with a trainable, low-dimensional codebook.

Q13. Which algorithms need feature scaling and which are indifferent — and why?

Scaling matters whenever an algorithm reads *distance* or *gradient* meaning into raw magnitudes: KNN, k-means, SVMs (kernel distances), PCA (variance directions), regularized linear models (an L2 penalty taxes small-scale features' coefficients harder), and neural nets (loss conditioning). Trees and tree ensembles are indifferent — splits are order-based, so any monotone transform leaves the split structure unchanged. Listing 3 supplies the disaster: one feature on a 500× scale drops unscaled KNN to 0.579 (near chance) because that feature owns every distance; any scaler restores ~0.88. The tell is "does the model compare features to each other by magnitude?"

Q14. Standard vs min-max vs robust scaling — what does each assume, and how do outliers affect them?

Standardization (z-score): mean 0, variance 1; the default, but both mean and especially σ are outlier-sensitive. *Min-max*: map to [0,1] by the observed range — bounded output for NNs, but range is the single most outlier-fragile statistic, so one absurd value crushes everyone into a sliver. *Robust*: subtract median, divide by IQR — immune to anything outside the middle 50%. Listing 3 quantifies it: with 2% gross outliers, standard and min-max leave the inliers' IQR at 0.031 and 0.0036 (the outliers inflated the divisor, shrinking the real data's working scale 50–300×), while robust holds IQR = 1.0 by construction. A scaler fit on contaminated statistics silently re-weights features.

Q15. What's the difference between column scaling and sklearn's `Normalizer`?

They operate on different axes. A `StandardScaler`/`MinMaxScaler`/`RobustScaler` rescales each *column* (feature) using statistics across rows. `Normalizer` rescales each *row* to unit norm — used for cosine-similarity text vectors so document length doesn't dominate. Confusing the two is a small but telling error: normalizing rows when you meant to standardize columns destroys per-feature comparability and vice versa. Also mind sparsity: centering (subtracting a mean) densifies sparse data, so sparse inputs want `MaxAbsScaler` or no-centering.

Q16. Filter, wrapper, and embedded feature selection — trade-offs and each family's characteristic failure.

Filter (correlation, ANOVA F, mutual information, chi-square) scores each feature against the target with no model — fast and parallel, but blind to interactions (a feature useless alone but decisive in combination scores zero) and to redundancy (ten copies all score high, it keeps all ten). *Wrapper* (forward/backward, RFE) trains the model on subsets — interaction-aware and model-specific, but expensive and prone to overfitting the validation set through search. *Embedded* (L1/Lasso, tree importances, permutation importance) gets selection as a fitting side effect — cheap and model-aware. Listing 4 (6 informative, 6 redundant, 18 noise): MI-filter, RFE, and RF importances each recover 8/8 real features; the failure signature is what matters — filters keep redundant copies, Lasso arbitrarily picks one of a correlated group, importances split credit across copies.

Q17. In Listing 4, L1 recovered only 6/10 real features. Was L1 wrong?

No — it exposed its known behavior. Lasso picks *one* feature from a correlated group and zeros the rest (Chapter 5's geometry and instability), so it correctly discarded some *redundant* copies — but at strong regularization it also let four noise features slip through while zeroing real-but-redundant ones. The lesson isn't "L1 is bad"; it's that L1 optimizes for a sparse predictive set, not for recovering every ground-truth feature, and among correlated features its choice is arbitrary and unstable across resamples. Naming that signature is the point, not the raw count.

Q18. Why is selecting features on the full dataset before cross-validation the most dangerous leak in high-dimensional work?

Because selection is a fitted transform, and fitting it on all rows (test included) chooses features partly for their correlation with test labels — then CV "validates" on the very labels that guided the choice. With many candidates and few rows (genomics: 20,000 genes, 100 patients) you can reach near-perfect cross-validated accuracy on *pure noise*. The fix is mechanical: selection lives inside the `Pipeline` and is refit per fold, so each fold's features are chosen without seeing that fold's labels. Same rule as imputation, scaling, and resampling — fit on training folds only.

Q19. Permutation importance vs impurity (Gini) importance — why prefer permutation?

Impurity importance ranks features by total split-impurity reduction, but it's biased toward high-cardinality and continuous features (more candidate split points = more chances to reduce impurity by chance) and is computed on training data, so it rewards overfitting. Permutation importance shuffles one column and measures the score drop on *held-out* data — model-agnostic, evaluated on validation, and immune to the cardinality bias. Its cost is one extra scoring pass per feature, and it can under-credit features within a correlated group (shuffling one, the model leans on its twin), so read it alongside the redundancy structure.

Q20. With 3% positives, why is accuracy the wrong metric, and what do you use instead?

A model predicting "negative" always scores 97% accuracy while catching zero positives — accuracy rewards the majority and is blind to the minority you actually care about. Use metrics tied to the positive class: PR-AUC (precision-recall area, which ignores the easy true negatives), recall at a fixed precision, or F-beta at a chosen operating point (Chapter 10). Fixing the metric comes *first* — before touching resampling — because the metric determines whether imbalance handling even helps.

Q21. Class weights, undersampling, oversampling, SMOTE, threshold moving — what does each actually buy?

Class weights reweight the loss (a positive counts $1/\pi$ as much) — one line, no data change, the cheapest defensible default for hard predictions. *Undersampling* drops majority rows — fast, but discards real information. *Oversampling* duplicates minority rows — no new information, overfitting risk. *SMOTE* synthesizes minority points by interpolating between minority neighbors — helps most under extreme scarcity with simple boundaries. *Threshold moving* trains as-is and moves the decision threshold off the never-sacred 0.5 to where the precision/recall trade-off belongs — often the rightest and least-used answer. Listing 5's ranking (PR-AUC): baseline 0.768 beat class weights 0.748, oversample 0.749, SMOTE 0.708, undersample 0.670.

Q22. In Listing 5, doing nothing beat every resampling method. Why?

PR-AUC is a *ranking* metric — it cares only about the order of scores — and the baseline random forest already orders positives above negatives fine. Resampling and reweighting change *threshold geometry* and probability calibration (where scores concentrate), which a ranking metric ignores; meanwhile undersampling threw away 92% of the majority (real cost) and SMOTE's linear interpolations plant synthetic positives on segments between true positives, landing some in majority territory in a 12-D overlapping space. The honest summary: imbalance handling mostly buys a usable default threshold and calibrated hard predictions; if you evaluate by ranking and tune the threshold downstream, baseline + threshold moving is very hard to beat.

Q23. Explain the SMOTE-before-split trap and where SMOTE must actually go.

SMOTE before the train/test split interpolates between minority points that then land on *different* sides of the split, so synthetic training points are literal blends of test rows — and the rebalanced "test set" no longer resembles the 3% world the model will face. Listing 5 measures the fantasy: PR-AUC **0.999** before-split vs 0.708 done honestly. SMOTE (and any sampler) must live *inside* the CV loop, after splitting, applied to training folds only — `imblearn.pipeline.Pipeline` exists precisely so samplers apply to train folds and not to validation. A suspiciously beautiful number on imbalanced data should be audited for this bug first.

Q24. Why do univariate outlier rules miss the outliers that damage models most?

Because the most damaging outliers are *leverage* points — extreme in the joint structure but unremarkable in any single column. Listing 6 corrupts 2% of rows with extreme *x* paired with wrong *y*; each value is marginally ordinary (an *x* of 8 is just a big *x*, a *y* near 0 is a typical *y*), so *z*-score on *y* flags 1/40 and IQR fences 2/40. Isolation Forest, seeing the joint (*x*, *y*) sitting far from the data's line, catches 32/40. Leverage is a *relationship* between columns, not a property of one, so column-at-a-time rules are structurally blind to it — check jointly whenever columns should covary.

Q25. Winsorizing vs deleting vs robust loss — compare, and say when deletion is dangerous.

Listing 6's corruption drags OLS's slope from the true 2.0 to 0.897. *Winsorizing* x at the 1st/99th percentiles reaches only 0.941 — it pulls extreme x 's inward but keeps the wrong y 's, now at moderate leverage: it treats a value, not a row, and a corrupted row keeps lying through its other columns. *Deleting* flagged rows recovers 1.668; *Huber loss* (quadratic near zero, linear in the tails, bounding gross residuals) recovers 1.759 with no detection step at all. Deletion is right when flagged points are *errors* (sensor glitches, bad joins) and dangerous when they're *real but rare* — a fraud model deleting its fraud cases, or dropping the largest customer in a revenue model. When unsure, bound influence instead of destroying data: robust losses, robust scalers, rank/log transforms.

Q26. When do you reach for a log transform, binning, or an interaction feature?

Each buys a linear model a specific nonlinearity (Listing 7). *Log* when the relationship or the feature is multiplicative/right-skewed — it turns multiplicative truth linear (R^2 0.603 $\rightarrow$ 0.973) and compresses heavy tails. *Binning* (discretize into buckets, then one-hot) when the response is non-monotone in a feature — a U-shaped truth invisible to a line (R^2 -0.001) becomes a step function a linear model can fit (0.889), at the cost of lost resolution and a bin-count hyperparameter. *Interactions* (add products like $x_1 \cdot x_2$) when the effect of one feature depends on another — an XOR-style truth is a coin flip additively (0.493) and near-perfect with one product column (0.998). Trees discover all three implicitly; linear models need them handed over.

Q27. A feature has 900 categories, each with real signal. Rank one-hot, hashing, frequency, and smoothed target encoding.

Listing 8 (≈ 9 rows/category): smoothed target encoding wins in a single column (0.734), edging 900-column one-hot (0.727) while being far narrower and not fragmenting trees. Hashing into 32 buckets pays a collision tax (0.625) because distinct categories share columns. Frequency encoding fails here by construction (0.619) — popularity was built to carry no signal. The general lesson: at high cardinality one-hot is wide and fragments trees, hashing trades accuracy for fixed memory, frequency only works if popularity is predictive, and smoothed/OOF target encoding gives the most signal per column — provided it's fit on train only with shrinkage for rare categories.

Q28. "Which model are you feeding?" — summarize how that answer reorganizes all feature engineering.

It's the first question of any feature-engineering answer because tree ensembles and linear models have nearly opposite needs. *Trees* are indifferent to monotone transforms, scaling, and binning, handle NaN natively (modern ones), and can split on raw categoricals (LightGBM/CatBoost) — their real needs are cross-variable interactions (which depth supplies) and signal-bearing encodings for huge cardinalities. *Linear models* need everything: scaling for the penalty, log/bins/splines for nonlinearity, explicit interaction columns, one-hot or target encodings, and imputation. Naming the model first tells you which two-thirds of the preprocessing menu you can skip — and which leakage traps (target encoding, SMOTE, selection) still apply to both regardless.